

Ziplink — URL Shortener Dashboard

Project Working Notes & Code Flow

1. Tech Stack

Core

Tech	Version	Purpose
React	19	UI library
React Router DOM	7	Client-side routing
Create React App	5	Build tooling
Bootstrap	5.3.3	CSS grid, tables, modals, pagination, dropdowns
Bootstrap Icons	1.11.3	All UI icons
Google Fonts	—	Poppins (primary), Inter (fallback)

External APIs

API	Endpoint	Used For
Shlink (self-hosted)	POST /rest/v3/short-urls	Creating short URLs
Google Favicons	//www.google.com/s2/favicons	Link row favicons
QR Server	//api.qrserver.com/v1/create-qr-code	Generating QR code images

Infrastructure

Service	Tool	Details
Docker	Shlink	shlinkio/shlink:stable, port 8080, SQLite, API key my-api-key-123
Storage	localStorage	User accounts, session, per-user links (no backend DB)

2. File Structure

```
demo/
|-- docker-compose.yml      # Shlink container config
|-- package.json            # Dependencies & scripts
|-- public/
|   '-- index.html          # HTML entry, CDN links, Bootstrap
'-- src/
    |-- index.js            # React entry: BrowserRouter + App
    |-- App.js              # Root router with auth guard
    |-- auth.js             # localStorage auth service
    |-- shlink.js           # Shlink API client
    |-- Dashboard.js        # Main app (all features)
    |-- Login.js            # Login page
    |-- Register.js         # Register page
    |-- auth.css            # Login/Register styles
```

```

|-- dashboard.css          # Dashboard styles + QR skeleton
'-- ...                    # CRA boilerplate (index.css, etc.)

```

3. Route Map

Path	Component	Auth Needed	Behavior
/login	Login	No	Login form
/register	Register	No	Signup form
/dashboard	Dashboard	Yes	Main URL shortener
*	—	No	Redirects to /dashboard

Auth guard (ProtectedRoute in App.js): checks `isAuthenticated()` -> if false, redirects to `/login`.

4. Auth Flow (auth.js)

Data in localStorage

Key	Format	Purpose
shlink_users	[{email, password}, ...]	Registered users
shlink_session	{"email": "..."} Current session	

Register

```

Register.js -> registerUser(email, pw)
  -> reads localStorage["shlink_users"]
  -> checks duplicate email
  -> pushes {email, pw} and saves
  -> returns {ok: true/false, error?: string}

```

Login

```

Login.js -> loginUser(email, pw)
  -> reads localStorage["shlink_users"]
  -> finds matching email + password
  -> writes localStorage["shlink_session"] = {email}
  -> returns {ok: true/false, error?: string}

```

Session Check

```

getCurrentUser() -> parses localStorage["shlink_session"] -> user object or null
isAuthenticated() -> !!getCurrentUser()

```

Logout

```

logoutUser() -> removes localStorage["shlink_session"]

```

5. Link Storage (Per-User Isolation)

Each user's links are stored under a unique localStorage key:

```
getLinksKey(email) -> "shlink_links_<email>"  
e.g. "shlink_links_alice@gmail.com"
```

Link Object Shape

```
{  
  "short": "http://192.168.1.15:8080/abc123",  
  "long": "https://example.com/page",  
  "title": "Page Title",  
  "date": "2026-06-07T14:30:00.000Z"  
}
```

CRUD Flow

- **Create:** API shortens -> preview modal -> user confirms -> `setLinks()` -> `useEffect` persists to localStorage
 - **Read:** `useState` initializer loads from localStorage on mount
 - **Update:** Edit modal -> `handleEdit()` maps over array -> `useEffect` persists
 - **Delete:** `handleDelete()` filters array -> `useEffect` persists
-

6. All Features — How They Work

6.1 URL Shortening

1. User pastes URL -> clicks "+ Add Link"
2. `handleShorten()` checks 5-link limit (skips for prodemo@gmail.com)
3. Calls `shlink.js` -> `shortenUrl()` -> `POST /rest/v3/short-urls` with `findTitle: true`
4. Shlink returns `{ shortUrl, longUrl, title }`
5. Title cleaned (`cleanTitle()`): strips HTML, collapses whitespace)
6. Short URL normalized to always use 192.168.1.15:8080 base
7. **PreviewModal** opens — user can edit title/URL before saving
8. On confirm -> prepended to `links` array with ISO date -> persisted to localStorage

6.2 Preview & Confirm

- Modal shows editable Title + Original URL fields
- Save -> `handleConfirm()` creates `{ short, long, title, date }` and prepends to list
- Cancel -> discards, closes modal

6.3 Link Table

- Columns: Title (favicon + truncated title + domain), Short Link (clickable), Added On (formatted date), Actions
- Clicking a row -> highlights it (`.table-primary`) -> shows QR in sidebar
- Clicking outside the table -> clears selection

6.4 Search

- Filters by `title.toLowerCase().includes(query)` OR `long.toLowerCase().includes(query)`
- Resets to page 1 on every keystroke

6.5 Sort

- Dropdown: Newest First (default) / Oldest First
- Sorts by `date` field

6.6 Pagination

- `PER_PAGE` = 3 links per page
- Previous / page numbers / Next buttons
- “Show All” expands to full list, button toggles to “Paginate”
- Display text: “Showing X to Y of Z links”

6.7 Edit Link

- Pencil icon -> `EditModal` with title + URL fields
- Save -> `handleEdit()` replaces matching `short` entry in array

6.8 Delete Link

- Trash icon -> `handleDelete()` filters out matching `short` entry

6.9 QR Code — Sidebar

- Click table row -> `selectedLink` set
- Sidebar card shows: favicon, title, QR image (150×150 via `api.qrserver.com`), short URL
- **Skeleton loader**: 15 violet squares pulse with staggered delays while image loads
- Placeholder: “Click a link to show QR” when nothing selected

6.10 QR Code — Modal

- QR button in table row -> `qrLink` set -> `QrModal` opens
- Shows spinner while 200×200 image loads
- Close button or backdrop click dismisses

6.11 5-Link Limit & Pro Modal

- Non-pro users (`email !== 'prodemo@gmail.com'`) limited to 5 links
- On 6th attempt -> Pro modal appears:
 - Stars icon, title “Upgrade to Pro”, message about limit
 - “Subscribe Now — \$4.99/mo” (UI only, no-op)
 - “Maybe later” dismisses

6.12 Profile Dropdown

- Avatar circle shows first letter of email
- Bootstrap dropdown: email, plan label (“Pro Plan” with crown / “Basic Plan”), divider, Logout
- `data-bs-toggle="dropdown"` (Bootstrap JS handles toggle)

6.13 Logout

- `handleLogout()` -> `logoutUser()` removes session -> navigate to `/login`
-

7. External API Integration

Shlink (only create — no read/update/delete)

```
POST http://192.168.1.15:8080/rest/v3/short-urls
Headers: X-API-Key: my-api-key-123, Content-Type: application/json
Body: { longUrl: "...", findTitle: true }
Response: { shortUrl: "...", longUrl: "...", title: "..." }
```

Google Favicons

```
GET https://www.google.com/s2/favicons?domain=<domain>&sz=<16|32>
```

QR Server

```
GET https://api.qrserver.com/v1/create-qr-code/?size=<WxH>&data=<encodedURL>
```

8. Skills Used

Skill	Where
React Hooks — <code>useState</code> , <code>useEffect</code>	All components
React Router — <code>Routes</code> , <code>Route</code> , <code>Navigate</code> , <code>useNavigate</code> , <code>Link</code> , <code>BrowserRouter</code>	<code>App.js</code> , <code>Login.js</code> , <code>Register.js</code>
Conditional rendering — <code>{condition && ...}</code> , <code>ternary, {condition ? ... : ...}</code>	<code>Dashboard.js</code> (modals, table, pagination, sidebar)
Controlled form inputs — <code>value</code> + <code>onChange</code>	Add URL, search, edit modals
localStorage — <code>getItem</code> , <code>setItem</code> , <code>removeItem</code>	<code>auth.js</code> , <code>Dashboard.js</code>
Async/await — API calls with <code>try/catch/finally</code>	<code>handleShorten()</code>
Array methods — <code>filter</code> , <code>map</code> , <code>sort</code> , <code>find</code> , <code>slice</code> , <code>push</code>	<code>Dashboard.js</code> (CRUD, search, sort, pagination)
Bootstrap 5 — grid (<code>row/col-lg-*</code>), cards, tables, modals, pagination, dropdowns, forms, spinner	All UI
Bootstrap Icons — 10+ icons used	Throughout
CSS animations — <code>@keyframes</code> , <code>animation-delay</code>	QR skeleton (<code>qrPulse</code>)
CSS gradients — <code>linear-gradient(135deg, #7C3AED, #3B82F6)</code>	All buttons, icons, active states
CSS transitions — hover effects on buttons, inputs	<code>auth.css</code> , <code>dashboard.css</code>
Media queries — <code>@media(max-width: 768px)</code> for mobile	<code>auth.css</code>
SVG — inline SVG with <code><rect></code> elements for QR skeleton	<code>Dashboard.js</code>
URL parsing — <code>new URL()</code> , <code>hostname</code> , <code>pathname</code>	<code>extractDomain()</code> , <code>normalizeShortUrl()</code>
Date formatting — <code>toLocaleDateString('en-GB', ...)</code>	Table “Added On” column
Docker Compose — Shlink container with SQLite volume	<code>docker-compose.yml</code>
Error handling — <code>try/catch</code> , <code>onError</code> on images, optional chaining	Throughout
useEffect cleanup patterns — dependency arrays, derived state reset	<code>Dashboard.js</code>

9. Key Constants

Constant	Value	Location
SHLINK_BASE	http://192.168.1.15:8080	shlink.js, Dashboard.js
API_KEY	my-api-key-123	shlink.js
PER_PAGE	3	Dashboard.js
USERS_KEY	shlink_users	auth.js
SESSION_KEY	shlink_session	auth.js
Pro user email	prodemo@gmail.com	Dashboard.js (hardcoded)

10. Component Tree

```

<BrowserRouter>                                (index.js)
  <App>                                           (App.js)
    |-- /login -> <Login />                      (Login.js)
    |-- /register -> <Register />                (Register.js)
    '-- /dashboard ->
      <ProtectedRoute>                          (App.js - auth guard)
        <Dashboard>                             (Dashboard.js)
          |-- <QrModal />                        (full-screen QR popup)
          |-- <PreviewModal />                  (title/URL edit before save)
          |-- <EditModal />                     (title/URL edit for saved links)
          |-- Pro Modal                         (inline conditional)
          |-- Navbar (Ziplink + profile dropdown)
          |-- Add-link form card
          |-- Search & sort bar
          |-- Links table
          |-- Pagination
          '-- Sidebar
            |-- Total Links card
            '-- QR Code card

```

11. Data Flow Diagram

User Action		Component		Storage/API
-----		-----		-----
Register form submit	->	Register.js	->	localStorage["shlink_users"]
Login form submit	->	Login.js	->	localStorage["shlink_session"]
Paste URL + click Add	->	Dashboard.js	->	Shlink API (POST /short-urls)
		PreviewModal shown	<-	{ shortUrl, longUrl, title }
User confirms preview	->	Dashboard.js	->	localStorage["shlink_links_{email}"]
Load dashboard	->	Dashboard.js	<-	localStorage["shlink_links_{email}"]
Edit link	->	Dashboard.js	->	localStorage["shlink_links_{email}"]
Delete link	->	Dashboard.js	->	localStorage["shlink_links_{email}"]
Search / sort / page	->	Dashboard.js	->	(computed in-memory from state)
Click table row	->	Dashboard.js	->	setSelectedLink -> QR sidebar
Logout	->	Dashboard.js	->	remove localStorage["shlink_session"]

12. Security Notes (Important for Demo Context)

- **Passwords stored in plaintext** in localStorage — not suitable for production
- **Session** is just an email string in localStorage — no tokens, no expiry
- **API key** is hardcoded in `shlink.js` — visible in browser DevTools
- **Per-user isolation** is convention-based (key naming) — not real isolation
- **All data** is in the browser's localStorage — clearing it loses everything
- **No server-side validation** — the app trusts client-side data entirely